

Documentation for Landon’s RT-TDDFT Photochemistry Thing

Landon Johnson

June 24, 2025

Contents

1	How to Use It	1
1.1	Downloading the Software	1
1.2	Obtaining an Optimized Geometry	2
1.3	Running the Code	2
1.3.1	INCAR	2
1.3.2	PHOTCAR	3
1.3.3	Setting $\hat{n}$	3
1.3.4	Setting up <code>twoLevelSystem.sh</code>	6
1.3.5	The Job Submission File	6
1.3.6	Batch Job Submission	6
2	Analyzing Output Data	7

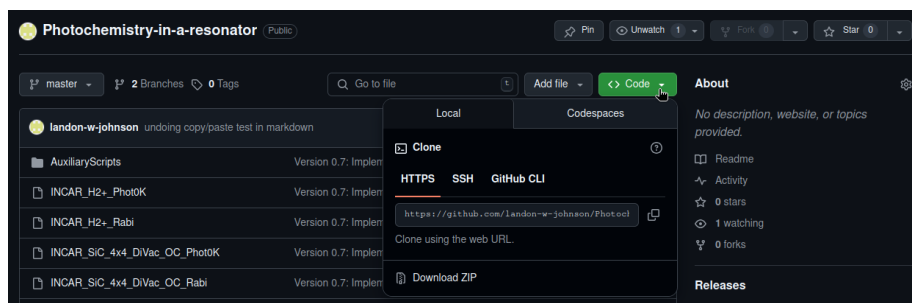
1 How to Use It

1.1 Downloading the Software

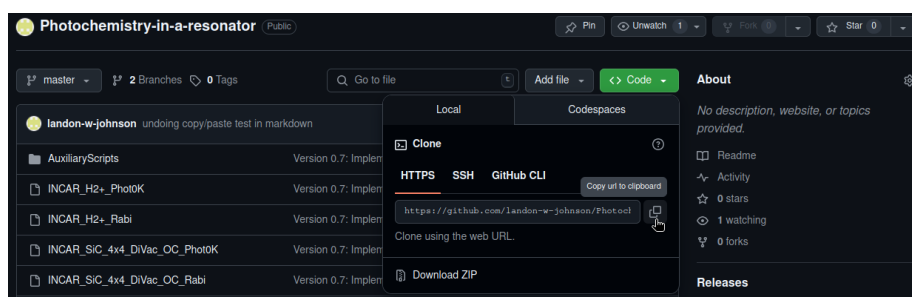
The software can be downloaded from my git repository at <https://github.com/landon-w-johnson/Photochemistry-in-a-resonator/tree/master>. The software can be downloaded manually from the webpage or the repo can be cloned. To clone the repo from the terminal, run

```
git clone https://github.com/landon-w-johnson/Photochemistry-in-a-  
resonator.git
```

This will create a directory wherever you are and download the latest “stable” version of the codes. **Note:** this code block did not copy/paste correctly when I tried it. The hyphens were slightly different than the URL. To get a copy/pastable address, go to the webpage and click the code button as shown below



and then click on the copy button as shown below



Note: Make sure you copy the **HTTPS** URL. If you're on a computer cluster, it may complain that **git** isn't a command or something. If this happens, run

```
module load git
```

and try again.

1.2 Obtaining an Optimized Geometry

This can be done as normal in VASP. At present, the code is only equipped to mess with the spin-up states in a spin-polarized calculation, so ensure that you set **ISPIN=2** in your **INCAR**. The code is also currently built on the assumption that you are using **ISYM=0**. You will need a **WAVECAR** file to start running the code, so you might as well set **LWAVE=.TRUE.** as well. **NOTE:** You must use only a single k-point at the moment.

1.3 Running the Code

This code makes use of a file called **PHOTCAR** that contains instructions for the custom code in the same syntax as **INCAR**. To keep things clearly distinguished, these settings will be broken down into subsections.

1.3.1 INCAR

Ensure that you have these additional tags in your **INCAR**:
ALGO=Damped

```
LDIAG=.FALSE.
TIME=0.1
NELMDL=2
```

These can also be present during your optimization. Feel free to play with the `TIME` variable. `NELMDL` can also be changed, but `NELMDL=0` may cause issues. I haven't tested that in a while. The orbitals did not seem to be consistent before and after running VASP without these tags. You will also need to set `NSW=0` if you're doing calculations with frozen nuclei or `NSW=1` if you're allowing nuclear motion. You must set `ISMear=-2` so that the electronic occupancies are read from `WAVECAR`. It is also advisable to set `ISTART=2` even though it hypothetically should be equivalent to the default `ISTART=1` when there's a `WAVECAR` present.

1.3.2 PHOTCAR

This file specifies initial conditions and other instructions for how the code should run. At present, the amplitude of the driving electric field within the resonator is given by

$$\vec{E}(t) = E_0 \cos(\omega t) \hat{n} \quad (1)$$

E is E_0 and `omega` is ω from equation 1. $\hat{n}$ is set within the FORTRAN scripts, but more on that later. `timeStep` is the time step in femtoseconds. **Make sure this matches with POTIM in your INCAR if you have NSW=1!** `RWA` is a boolean that determines whether or not to use the rotating wave approximation (RWA). `itScheme` is an integer that sets the iteration scheme. `itScheme=1` will use Euler, `itScheme=2` will use Verlet, `itScheme=3` will use second order Runge-Kutta, and `itScheme=4` will use fourth order Runge-Kutta. `bands` is a space-separated array of the orbital indices whose excitation is tuned to the resonator. At present, this must be precisely two bands in ascending order. `CHGint` is an integer that determines how often to save the CHG files from VASP. For example, `CHGint=12` will save the CHG file every 12th time step. `CHGint=0` means that no CHG files will be saved.

1.3.3 Setting $\hat{n}$

At present, this must be hard-coded into `firstTimeStep.f` and `updateWAVECAR.f`. Further, it can only point in the direction of one of the lattice vectors defined in your `POSCAR`. This is one of the next things that I will update, because I'm tired of having to manually do this every time I need to change $\hat{n}$. In order to find which way you ought to make $\hat{n}$ point, just run this program for at least one time step and examine the contents of `outputFile.txt`. At the very top of the file, you should see values for `px`, `py`, and `pz` as in the example below.

```

CHGInt = 0

E = 1.0000000000000001E-005
timeStep = 1.000000000000000E-002
omega = 1.1083460000000001E-001
RWA = F
itScheme = 1
bandArray =      7      8
numPlaneWaves =      4886.0000
npw = 4886

px(7,8) = ( 0.000000000000000E+000 , -1.353935777594524E-001 )
px(8,7) = ( 0.000000000000000E+000 , 1.353935777594524E-001 )
py(7,8) = ( 0.000000000000000E+000 , 1.782141974642879E-004 )
py(8,7) = ( 0.000000000000000E+000 , -1.782141974642879E-004 )
pz(7,8) = ( 0.000000000000000E+000 , -2.730963503447424E-009 )
pz(8,7) = ( 0.000000000000000E+000 , 2.730963461885211E-009 )
omega_0 = 0.11083458553812
d_bCoef(8) = ( -0.000000000000000E+000 , -1.221582388765144E-005 )

```

You probably want to set it to whichever has the highest value. Here are the locations that need to be set in `firstTimeStep.f`:

772	IF (RWA) THEN	794	ELSE IF (itScheme.EQ.2) THEN ! Verlet
773	d_bCoef(bandArray(2)) = (q*E/(2*m*hbar*omega_0))*	795	ALLOCATE(d2_bCoef(nb))
774	\$ pz(bandArray(2),bandArray(1))	796	IF (RWA) THEN
775	ELSE	797	PRINT *, "Solving for d^2/dt under RWA"
776	d_bCoef(bandArray(2)) = (q*E/(m*hbar*omega_0))*	798	d2_bCoef(bandArray(1)) = -(q**2)*(E**2)*
777	\$ pz(bandArray(2),bandArray(1))	799	\$ pz(bandArray(1),bandArray(2))*
778	END IF	800	\$ pz(bandArray(2),bandArray(1))/
		801	\$ (4*m**2*hbar**2*omega_0**2)
		802	d2_bCoef(bandArray(2)) = (q*E*i*(omega-omega_0)*
		803	\$ pz(bandArray(2),bandArray(1)))/
		804	\$ (2*m*hbar*omega_0)
		805	ELSE
		806	PRINT *, "Solving for d^2/dt without RWA"
		807	d2_bCoef(bandArray(1)) = -(q**2)*(E**2)*
		808	\$ pz(bandArray(1),bandArray(2))*
		809	\$ pz(bandArray(2),bandArray(1))/
		810	\$ ((m**2)*(hbar**2)*(omega_0**2))
		811	d2_bCoef(bandArray(2)) = (q*E*i*m)/(m*hbar)
831	ELSE IF (itScheme.EQ.3) THEN ! Runge-Kutta 2nd Order	856	ELSE IF (itScheme.EQ.4) THEN ! Runge-Kutta 4th Order
832	rk(1,1) = dt*d_bCoef(bandArray(1))	857	rk(1,1) = dt*d_bCoef(bandArray(1))
833	rk(2,1) = dt*d_bCoef(bandArray(2))	858	rk(2,1) = dt*d_bCoef(bandArray(2))
834	IF (RWA) THEN	859	IF (RWA) THEN
835	rk(1,2) = dt*((-q*E*EXP(im*(omega-omega_0)*dt)*	860	rk(1,2) = dt*((-q*E*EXP(im*(omega-omega_0)*dt/2)*
836	\$ pz(bandArray(1),bandArray(2))*(rk(2,1))	861	\$ pz(bandArray(1),bandArray(2))*(rk(2,1)/2)
837	\$ /(2*m*hbar*omega_0)	862	\$ /(2*m*hbar*omega_0)
838	rk(2,2) = dt*((q*E*EXP(im*(omega_0-omega)*dt)*	863	rk(2,2) = dt*((q*E*EXP(im*(omega_0-omega)*dt/2)*
839	\$ pz(bandArray(2),bandArray(1))*(1+rk(1,1))/	864	\$ pz(bandArray(2),bandArray(1))*(1+rk(1,1)/2)/
840	\$ (2*m*hbar*omega_0)	865	\$ (2*m*hbar*omega_0)
841	ELSE	866	rk(1,3) = dt*((-q*E*EXP(im*(omega-omega_0)*dt/2)*
842	rk(1,2) = dt*((-q*E*DCOS(omega*dt)*	867	\$ pz(bandArray(1),bandArray(2))*(rk(2,2)/2)
843	\$ pz(bandArray(1),bandArray(2))*	868	\$ /(2*m*hbar*omega_0)
844	\$ EXP(-im*omega_0*dt)*rk(2,1))/	869	rk(2,3) = dt*((q*E*EXP(im*(omega_0-omega)*dt/2)*
845	\$ (m*hbar*omega_0)	870	\$ pz(bandArray(2),bandArray(1))*(1+rk(1,2)/2)/
846	rk(2,2) = dt*((q*E*DCOS(omega*dt)*	871	\$ (2*m*hbar*omega_0)
847	\$ pz(bandArray(2),bandArray(1))*	872	rk(1,4) = dt*((-q*E*EXP(im*(omega-omega_0)*dt)*
848	\$ EXP(im*omega_0*dt)*(1+rk(1,1))/	873	\$ pz(bandArray(1),bandArray(2))*rk(2,3)/
849	\$ (m*hbar*omega_0)	874	\$ (2*m*hbar*omega_0)
850	END IF	875	rk(2,4) = dt*((q*E*EXP(im*(omega_0-omega)*dt)*
		876	\$ pz(bandArray(2),bandArray(1))*(1+rk(1,3))/
		877	\$ (2*m*hbar*omega_0)

```

878 ELSE
879 rk(1,2) = dt*((-q*E*DCOS(omega*dt/2)*
880 pz(bandArray(1),bandArray(2))*
881 EXP(-im*omega_0*dt/2)*(rk(2,1)/2))/
882 (m*hbar*omega_0))
883 rk(2,2) = dt*((q*E*DCOS(omega*dt/2)*
884 pz(bandArray(2),bandArray(1))*
885 EXP(im*omega_0*dt/2)*(1+rk(1,1)/2))/
886 (m*hbar*omega_0))
887 rk(1,3) = dt*((-q*E*DCOS(omega*dt/2)*
888 pz(bandArray(1),bandArray(2))*
889 EXP(-im*omega_0*dt/2)*(rk(2,2)/2))/
890 (m*hbar*omega_0))
891 rk(2,3) = dt*((q*E*DCOS(omega*dt/2)*
892 pz(bandArray(2),bandArray(1))*
893 EXP(im*omega_0*dt/2)*(1+rk(1,2)/2))/
894 (m*hbar*omega_0))
895 rk(1,4) = dt*((-q*E*DCOS(omega*dt)*
896 pz(bandArray(1),bandArray(2))*
897 EXP(-im*omega_0*dt)*rk(2,3))/
898 (m*hbar*omega_0))
899 rk(2,4) = dt*((q*E*DCOS(omega*dt)*
900 pz(bandArray(2),bandArray(1))*
901 EXP(im*omega_0*dt)*(1+rk(1,3))/
902 (m*hbar*omega_0))
903 END IF

```

In the images shown, every instance of **pz** needs to be set to **px** to make $\hat{n}$ align with the first lattice vector in POSCAR, **py** to align with the second lattice vector, or **pz** to align with the third lattice vector. You can get away with only setting it in the appropriate iteration scheme and RWA/non-RWA code block, but this can cause issues if you end up changing your PHOTCAR. Likewise, you'll need to do the same process in the following sections of **updateWAVECAR.f**:

```

433 IF (RWA) THEN
434 d_bCoef(bandArray(1)) = -(q*E)/(2*m*hbar*omega_0)*
435 $ EXP(im*(omega-omega_0)*t)*
436 $ pz(bandArray(1),bandArray(2))*bCoef(bandArray(2))
437 $ d_bCoef(bandArray(2)) = ((q*E)/(2*m*hbar*omega_0))*
438 $ EXP(im*(omega_0-omega)*t)*
439 $ pz(bandArray(2),bandArray(1))*bCoef(bandArray(1))
440 ELSE
441 d_bCoef(bandArray(1)) = -(q*E/(m*hbar*omega_0))*
442 $ DCOS(omega*t)*pz(bandArray(1),bandArray(2))*
443 $ EXP(-im*omega_0*t)*bCoef(bandArray(2))
444 $ d_bCoef(bandArray(2)) = (q*E/(m*hbar*omega_0))*
445 $ DCOS(omega*t)*pz(bandArray(2),bandArray(1))*
446 $ EXP(im*omega_0*t)*bCoef(bandArray(1))
447 END IF
448 WRITE(*,'(A,I0.1,A,"( ",ES23.15E3," ", ",ES23.15E3," )"')')
449 $ 'd_bCoef(',bandArray(1),') = ', d_bCoef(bandArray(1))
450 WRITE(*,'(A,I0.1,A,"( ",ES23.15E3," ", ",ES23.15E3," )"')')
451 $ 'd_bCoef(',bandArray(2),') = ', d_bCoef(bandArray(2))
452
506 ELSE IF (itScheme.EQ.2) THEN
507 ALLOCATE(d2_bCoef(nb))
508 IF (RWA) THEN
509 d2_bCoef(bandArray(1)) =
510 $ -(q*E*pz(bandArray(1),bandArray(2))
511 $ *EXP(im*(omega-omega_0)*t)*
512 $ im*(omega-omega_0)*bCoef(bandArray(2)))/
513 $ (2*m*hbar*omega_0) -
514 $ ((q*2)*(E*2)*pz(bandArray(1),bandArray(2))
515 $ *pz(bandArray(2),bandArray(1))*bCoef(bandArray(1)))/
516 $ (4*(m*2)*(hbar*2)*(omega_0*2))
517 d2_bCoef(bandArray(2)) =
518 $ (q*E*im*(omega_0-omega)*
519 $ pz(bandArray(2),bandArray(1))*
520 $ EXP(im*(omega_0-omega)*t)*bCoef(bandArray(1)))/
521 $ (2*m*hbar*omega_0) -
522 $ ((q*2)*(E*2)*pz(bandArray(1),bandArray(2))*
523 $ pz(bandArray(2),bandArray(1))*bCoef(bandArray(2)))/
524 $ (4*(m*2)*(hbar*2)*(omega_0*2))
525 ELSE IF (itScheme.EQ.3) THEN
526 rk(1,1) = dt*d_bCoef(bandArray(1))
527 rk(2,1) = dt*d_bCoef(bandArray(2))
528 IF (RWA) THEN
529 rk(1,2) = dt*((-q*E*EXP(im*(omega-omega_0)*(t+dt))*
530 $ pz(bandArray(1),bandArray(2))*
531 $ (bCoef(bandArray(2))*rk(2,1)))/
532 $ (2*m*hbar*omega_0))
533 rk(2,2) = dt*((q*E*EXP(im*(omega_0-omega)*(t+dt))*
534 $ pz(bandArray(2),bandArray(1))*
535 $ (bCoef(bandArray(1))*rk(1,1)))/
536 $ (2*m*hbar*omega_0))
537 ELSE
538 rk(1,2) = dt*((-q*E*DCOS(omega*(t+dt))*
539 $ pz(bandArray(1),bandArray(2))*
540 $ EXP(-im*omega_0*(t+dt))*
541 $ (bCoef(bandArray(2))*rk(2,1)))/
542 $ (m*hbar*omega_0))
543 rk(2,2) = dt*((q*E*DCOS(omega*(t+dt))*
544 $ pz(bandArray(2),bandArray(1))*
545 $ EXP(im*omega_0*(t+dt))*
546 $ (bCoef(bandArray(1))*rk(1,1)))/
547 $ (m*hbar*omega_0))
548 END IF
549

```

```

535 ELSE IF (itScheme EQ 4) THEN
536   rk(1,1) = dt*d_bCoef(bandArray(1))
537   rk(2,1) = dt*d_bCoef(bandArray(2))
538   IF (RNA) THEN
539     rk(1,2) = dt*((-q+E*EXP(im*(omega-omega_0)*(t+dt/2))*
540   $   pz(bandArray(1),bandArray(2))*
541   $   (bCoef(bandArray(2))+rk(2,1)/2))/
542   $   (2*m*hbar*omega_0))
543     rk(2,2) = dt*((q+E*EXP(im*(omega_0-omega)*(t+dt/2))*
544   $   pz(bandArray(2),bandArray(1))*
545   $   (bCoef(bandArray(1))+rk(1,1)/2))/
546   $   (2*m*hbar*omega_0))
547     rk(1,3) = dt*((-q+E*EXP(im*(omega-omega_0)*(t+dt/2))*
548   $   pz(bandArray(1),bandArray(2))*
549   $   (bCoef(bandArray(2))+rk(2,2)/2))/
550   $   (2*m*hbar*omega_0))
551     rk(2,3) = dt*((q+E*EXP(im*(omega_0-omega)*(t+dt/2))*
552   $   pz(bandArray(2),bandArray(1))*
553   $   (bCoef(bandArray(1))+rk(1,2)/2))/
554   $   (2*m*hbar*omega_0))
555     rk(1,4) = dt*((-q+E*EXP(im*(omega-omega_0)*(t+dt))*
556   $   pz(bandArray(1),bandArray(2))*
557   $   (bCoef(bandArray(2))+rk(2,3))/
558   $   (2*m*hbar*omega_0))
559     rk(2,4) = dt*((q+E*EXP(im*(omega_0-omega)*(t+dt))*
560   $   pz(bandArray(2),bandArray(1))*
561   $   (bCoef(bandArray(1))+rk(1,3))/
562   $   (2*m*hbar*omega_0))
563 ELSE
564   rk(1,2) = dt*((-q+E*DCOS(omega*(t+dt/2))*
565   $   pz(bandArray(1),bandArray(2))*
566   $   EXP(-im*omega_0*(t+dt/2))*
567   $   (bCoef(bandArray(2))+rk(2,1)/2))/
568   $   (m*hbar*omega_0))
569   rk(2,2) = dt*((q+E*DCOS(omega*(t+dt/2))*
570   $   pz(bandArray(2),bandArray(1))*
571   $   EXP(im*omega_0*(t+dt/2))*
572   $   (bCoef(bandArray(1))+rk(1,1)/2))/
573   $   (m*hbar*omega_0))
574   rk(1,3) = dt*((-q+E*DCOS(omega*(t+dt/2))*
575   $   pz(bandArray(1),bandArray(2))*
576   $   EXP(-im*omega_0*(t+dt/2))*
577   $   (bCoef(bandArray(2))+rk(2,2)/2))/
578   $   (m*hbar*omega_0))
579   rk(2,3) = dt*((q+E*DCOS(omega*(t+dt/2))*
580   $   pz(bandArray(2),bandArray(1))*
581   $   EXP(im*omega_0*(t+dt/2))*
582   $   (bCoef(bandArray(1))+rk(1,2)/2))/
583   $   (m*hbar*omega_0))
584   rk(1,4) = dt*((-q+E*DCOS(omega*(t+dt))*
585   $   pz(bandArray(1),bandArray(2))*
586   $   EXP(-im*omega_0*(t+dt))*
587   $   (bCoef(bandArray(2))+rk(2,3))/
588   $   (m*hbar*omega_0))
589   rk(2,4) = dt*((q+E*DCOS(omega*(t+dt))*
590   $   pz(bandArray(2),bandArray(1))*
591   $   EXP(im*omega_0*(t+dt))*
592   $   (bCoef(bandArray(1))+rk(1,3))/
593   $   (m*hbar*omega_0))
594 END IF

```

1.3.4 Setting up twoLevelSystem.sh

This script controls the overall workflow of the software. All you should have to do in here is comment and uncomment a couple lines. If you're running this on CCAST, uncomment the lines that end in `# CCAST` and comment out the lines that end in `# NERSC`. If you're running this on NERSC, do the opposite.

1.3.5 The Job Submission File

The important thing here is that you tell your job submission file to execute `twoLevelSystem.sh`. **DO NOT** have your job submission file directly execute VASP or the FORTRAN scripts! There is a sample submission file for NERSC and CCAST in the repository. Once you have this set up, submit the job as you normally would, e.g.

```
qsub job.pbs
```

on CCAST or

```
sbatch submit-NERSC.sh
```

on NERSC.

1.3.6 Batch Job Submission

Under the directory `AuxiliaryScripts`, there is a script named `makeTrajectoryGrid.sh`. This script automates the process of submitting batches of jobs where you sweep over different initial conditions. You'll need to tell it the value of ω_0 , the resonant angular frequency defined by the energy difference between your upper and lower electronic states. To find it, run the program for at least one time

step and view the contents of `outputFile.txt`. At the very top, you should see a value of `omega_0` printed, as shown below.

```
CHGint = 0

E = 1.0000000000000001E-005
timeStep = 1.000000000000000E-002
omega = 1.1083460000000001E-001
RWA = F
itScheme = 1
bandArray =      7      8
numPlaneWaves =      4886.0000
npw = 4886
px(7,8) = ( 0.000000000000000E+000 , -1.353935777594524E-001 )
px(8,7) = ( 0.000000000000000E+000 , 1.353935777594524E-001 )
py(7,8) = ( 0.000000000000000E+000 , 1.782141974642879E-004 )
py(8,7) = ( 0.000000000000000E+000 , -1.782141974642879E-004 )
pz(7,8) = ( 0.000000000000000E+000 , -2.730963503447424E-009 )
pz(8,7) = ( 0.000000000000000E+000 , 2.730963461885211E-009 )
omega_0 = 0.11083458553812
d_bCoef(8) = ( -0.000000000000000E+000 , -1.221582388765144E-005 )
```

BASH doesn't like to use floating point (decimal) variables, so you'll have to feed it an integer. This integer is explicitly written after a decimal point in a string. **NOTE:** You'll have to manually include the appropriate amount of 0's in the decimal string at the location shown below in `makeTrajectoryGrid.sh`.

```
34 sed -i "s/E[[:space:]]*=[[:space:]]*\(\([0-9]\+\.[0-9]\+\)\|<tmp>\)/E = ${amplitude}/g" PHOTCAR
35 sed -i "s/omega[[:space:]]*=[[:space:]]*\(\([0-9]\+\.[0-9]\+\)\|<tmp>\)/omega = 0.0${omega}/g" PHOTCAR
36 sed -i "s/timeStep[[:space:]]*=[[:space:]]*\(\([0-9]\+\.[0-9]\+\)\|<tmp>\)/timeStep = ${dt}/g" PHOTCAR
37 sed -i "s/POTIM[[:space:]]*=[[:space:]]*\(\([0-9]\+\.[0-9]\+\)\|<tmp>\)/POTIM=${dt}/g" INCAR
```

In the example images shown above, the final 0 in line 35 needs to be removed, so that line 35 ends with `0.${omega}/g'' PHOTCAR`.

2 Analyzing Output Data

The main output data file is `dataFile.txt`. This is what you'll want for plotting purposes. All of the standard output of the programs being run is stored into `outputFile.txt`. This is what you'll generally want to look at for debugging purposes. The most sophisticated and user-friendly plotting script that I currently have is `make_all_plots.m`. It is set up to plot the results of batches of jobs. You'll need to tell it which directories to look in, e.g. `NoRWA` and `RWA` or only one of them. You'll also need to tell it the value of ω_0 , which should be the same value that you fed to `makeTrajectoryGrid.sh`, except it's actually a floating point variable here. Then you'll need to give it the value of the transition dipole moment, which should be whatever the value of `px`, `py`, or `pz` was for the first time step. Don't worry about saving it as `px` or `py` in the MATLAB script if that's what you used during the actual run. Save it as `pz` in the MATLAB script

regardless. Depending on your outer directories (**NoRWA** and/or **RWA**), you'll need to modify a couple lines that actually make the plot. These lines end in **%%%** **Manually edit this line!** so that you can find them easily. Just include the relevant legend entries for what you're plotting. If the conditions are right, you can also run **prepTemp&Freq.sh** to get time-dependent internuclear distance, temperature and/or molecular resonant frequency plots via **make_all_plots.m**.